Name: Deras Pangestu Gusti (08)
Class: SIB 1G
NIM: 254107060114

JOBSHEET XI
QUEUE

1. Learning Objectives

After completing this practical session, students will be able to:

1. Understand the structure and concept of the Queue data structure
2. Create and declare a Queue data structure
3. Implement Queue algorithms using arrays

2. Labs 1

2.1 Experiment 1 : Queue Basic Operations

Allocated Time : 90 minutes

In this experiment, we will implement the basic operations of the Queue algorithm. Take a look at the following Queue class diagram:

Queue
data: int[] front: int rear: int size: int max: int
Queue(n: int) isFull(): boolean isEmpty(): boolean enqueue(dt: int): void dequeue(): int peek: void print(): void clear(): void

The following steps will implement Java program from the above design.

2.1.1. Experiment Steps

1. Create a new folder named **P1Jobsheet11** or another prefered name, inside the Praktikum-ASD repository, then create a new class named **Queue**.
2. Add the attributes of the **Queue** as shown in the class diagram, then add the constructor as described below:

```
int[] data;
int front, rear, size, max;

public Queue(int n){
    max = n;
```

```
    data = new int[max];
    size = 0;
    front = rear = -1;
}
```

3. Create a boolean method named **isEmpty** to check whether the queue is empty.

```
boolean isEmpty(){
        if(size == 0){
            return true;
        }else{
            return false;
        }
    }
```

4. Create a boolean method named **isFull** to check whether the queue is full.

```
boolean isFull(){
        if(size == max){
            return true;
        }else{
            return false;
        }
    }
```

5. Create a void method named **peek** to display the element at the **front** of the queue.

```
void peek(){
        if(!isEmpty()){
            System.out.println("Front data: "+data[front]);
        }else{
            System.out.println("Queue is empty!!!");
        }
    }
```

6. Create a void method named **print** to display all elements in the queue, from the **front** to the **rear** position.

```
void print(){
        if(!isEmpty()){
            int i = front;
            while(i != rear){
                System.out.print(data[i]+" ");
                i = (i+1)%max;
            }
            System.out.println(data[i]+" ");
            System.out.println("Number of element: "+size);
        }else{
```

```
            System.out.println("Queue is empty!!!");
        }
    }
```

7. Create a void method named **clear** to remove all elements from the queue.

```
void clear(){
        if(!isEmpty()){
            front = rear = -1;
            size = 0;
            System.out.println("All data has been successfully
removed!");
        }else{
            System.out.println("Queue is already empty!!!");
        }
    }
```

8. Create a void method named **enqueue** to add an element to the queue, using a parameter **dt** of type **int**.

```
void enqueue(int dt){
        if(!isFull()){
            if(isEmpty()){
                front=rear=0;
            }else{
                if(rear==max-1){
                    rear=0;
                }else{
                    rear++;
                }
            }
            data[rear]=dt;
            size++;
            System.out.printf("%d is successfully added at index %d\n",
dt, rear);
        }else{
            System.out.println("Queue is full!!!");
        }
    }
```

9. Create an **int** method named **dequeue** to remove and return the data from the **front** position of the queue.

```
int dequeue(){
        int dt = 0;
        if(!isEmpty()){
            dt = data[front];
            size--;
```

```
            if(isEmpty()){
                front=rear=-1;
            }else{
                if(front==max-1){
                    front=0;
                }else{
                    front++;
                }
            }
        }else{
            System.out.println("Queue is empty!!!");
        }
        return dt;
    }
```

10. Next, create a new class named **QueueMain**. Create a **void** method named **menu** to display and handle the program's menu options during execution.

```
static void menu(){
        System.out.println("Available menu:");
        System.out.println("1. Enqueue");
        System.out.println("2. Dequeue");
        System.out.println("3. Print");
        System.out.println("4. Peek");
        System.out.println("5. Clear");
        System.out.println("=======================");
    }
```

11. Create the **main** method, then declare a **Scanner** object named **sc**.
12. Create a variable named **n** to store the user input representing the maximum number of elements that can be stored in the queue.

```
Scanner sc = new Scanner(System.in);
System.out.print("Input maximum number of data: ");
int n = sc.nextInt();
```

13. Instantiate a **Queue** object named **Q**, passing **n** as the queue's element capacity.

```
Queue Q = new Queue(n);
```

14. Declare an integer variable named **choice** to store the user's menu selection.
15. Use a **do-while** loop to keep the program running based on user input. Inside the loop, use a **switch-case** statement to execute queue operations according to the user's menu selection.

```
public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Input maximum number of data: ");
        int n = sc.nextInt();
        Queue Q = new Queue(n);
```

```
        int choice = -1;
        do {
            menu();
            choice = sc.nextInt();
            switch (choice) {
                case 1:
                    System.out.print("Input new data: ");
                    int newData = sc.nextInt();
                    Q.enqueue(newData);
                    break;
                case 2:
                    int outData = Q.dequeue();
                    if (outData != 0) {
                        System.out.println("Removed data: " + outData);
                        break;
                    }
                case 3:
                    Q.print();
                    break;
                case 4:
                    Q.peek();
                    break;
                case 5:
                    Q.clear();
                    break;
            }
        } while (choice==1 || choice==2 || choice==3 || choice==4 ||
choice==5);
    }
```

16. Compile and run the **QueueMain** class, then observe the results.

```java
package com.jobsheet11;

public class Queue8
{
    int[] data;
    int front,
        rear,
        size,
        max;

    public Queue8(int n)
    {
        max = n;
        data = new int[max];
        size = 0;
        front = rear = -1;
    }

    boolean isEmpty()
    {
        if (size == 0)
            return true;
        else
            return false;
    }

    boolean isFull()
    {
        if (size == max)
            return true;
        else
            return false;
    }

    void peek()
    {
        if (!isEmpty())
            System.out.println("Front data: " + data[front]);
        else
            System.out.println(x: "Queue is empty!!!");
    }

```

```
    void print()
    {
        if (!isEmpty())
        {
            int i = front;
            while (i != rear)
            {
                System.out.print(data[i] + " ");
                i = (i + 1) % max;
            }
            System.out.println(data[i] + " ");
            System.out.println("Number of element: " + size);
        }
        else
            System.out.println(x: "Queue is empty!!!");
    }

    void clear()
    {
        if (!isEmpty())
        {
            front = rear = -1;
            size = 0;
            System.out.println(x: "All data has been successfully removed!");
        }
        else
            System.out.println(x: "Queue is already empty!!!");
    }

```

```java
    void enqueue(int dt)
    {
        if (!isFull())
        {
            if (isEmpty())
                front = rear = 0;
            else
                if (rear == max - 1)
                    rear = 0;
                else
                    rear++;

            data[rear] = dt;
            size++;
            System.out.printf(format: "%d is successfully added at index %d\n", dt, rear);
        }
        else
            System.out.println(x: "Queue is full!!!");
    }

    int dequeue()
    {
        int dt = 0;
        if (!isEmpty())
        {
            dt = data[front];
            size--;

            if(isEmpty())
                front = rear = -1;
            else
                if (front == max - 1)
                    front = 0;
                else
                    front++;
        }
        else
            System.out.println(x: "Queue is empty!!!");

        return dt;
    }
}
```

```java
package com.jobsheet11;

import java.util.Scanner;

public class QueueMain
{
    static void menu()
    {
        System.out.print
        (
            """
            \nAvailable menu:
            1. Enqueue
            2. Dequeue
            3. Print
            4. Peek
            5. Clear
            0. Exit
            =======================
            """
        );
    }

    Run main | Debug main | Run | Debug
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.print(s: "Input maximum number of data: ");
        int n = sc.nextInt();

        Queue8 Q = new Queue8(n);

        int choice = -1;
        do
        {
            menu();
            choice = sc.nextInt();
```

```java
            switch (choice)
            {
                case 1:
                    System.out.print(s: "Input new data: ");
                    int newData = sc.nextInt();
                    Q.enqueue(newData);
                    break;

                case 2:
                    int outData = Q.dequeue();
                    if (outData != -1)
                    {
                        System.out.println("Removed data: " + outData);
                        break;
                    }

                case 3:
                    Q.print();
                    break;

                case 4:
                    Q.peek();
                    break;

                case 5:
                    Q.clear();
                    break;

                case 0:
                    sc.close();
                    return;

                default:
                    System.out.println(x: "Input Invalid!!!");
            }
        }
        while
        (
            choice == 1 || choice == 2 || choice == 3 ||
            choice == 4 || choice == 5
        );
    }
}
```

```
Input maximum number of data: 4

Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
0. Exit
=======================
1
Input new data: 15
15 is successfully added at index 0

Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
0. Exit
=======================
1
Input new data: 31
31 is successfully added at index 1

Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
0. Exit
=======================
3
15 31
Number of element: 2
```

```
Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
0. Exit
=======================
4
Front data: 15

Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
0. Exit
=======================
2
Removed data: 15

Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
0. Exit
=======================
3
31
Number of element: 1
```

2.1.2. Output Verification

Verify the output with the following:

```
Input maximum number of data: 4
Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
=======================
1
Input new data: 15
15 is successfully added at index 0
Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
```

```
=======================
1
Input new data: 31
31 is successfully added at index 1
Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
=======================
3
15 31
Number of element: 2
Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
=======================
4
Front data: 15
Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
=======================
2
Removed data: 15
Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
=======================
3
31
Number of element: 1
Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
=======================
```

2.1.3. Question

1. In the constructor, why are the initial values of the **front** and **rear** attributes set to **-1**, while the size attribute is set to **0**?

   In the constructor, the attributes front and rear are set to -1 to indicate that the queue is initially empty and no valid index is being used. The size attribute is set to 0 to represent that there are no elements stored in the queue at the start. This combination clearly distinguishes the empty state from any valid occupied state.

2. In the **Enqueue** method, explain the meaning and purpose of the following code snippet!

```
if (rear == max - 1) {
    rear = 0;
```

This code ensures the circular behaviour of the queue. When the rear pointer reaches the last index (max - 1), it wraps around to index 0. This allows the queue to reuse empty slots at the beginning of the array once elements have been dequeued, preventing wasted space and implementing a circular queue.

3. In the **Dequeue** method, explain the meaning and purpose of the following code snippet!

```
if (front == max - 1) {
    front = 0;
```

This code performs the same circular adjustment for the front pointer. When the front reaches the last index of the array, it wraps back to index 0. This ensures that the queue can continue to remove elements correctly even after the pointer has reached the end of the array, maintaining the circular structure.

4. In the **print** method, why does the loop variable **i** start from **front** instead of **0** (i.e., int i = 0)?

   The loop variable i starts from front instead of 0 because the queue does not necessarily begin at index 0. After several enqueue and dequeue operations, the logical start of the queue may be at a different position. Starting from front guarantees that the elements are printed in the correct logical order, not simply from the beginning of the array.

5. Review the **print** method again, and explain the meaning of the following code snippet!

```
i = (i + 1) % max;
```

   This expression increments the index i and uses the modulo operator to wrap it around when it reaches max. It ensures that the traversal continues correctly in a circular manner, moving from the end of the array back to the beginning without exceeding array bounds.

6. Show the code snippet that represents a **queue overflow**!

```
else
    System.out.println(x: "Queue is full!!!");
```

7. When a queue overflow or underflow occurs, the program continues to run and only displays informational text. Modify the program so that it stops when a queue overflow or underflow happens!

```
else
{
    System.out.println(x: "Queue is full!!!");
    return;
}
```

```
else
{
    System.out.println(x: "Queue is empty!!!");
    return -1;
}
```

2.2. Experiment 2 : Academic Service Queue

Allocated Time : 90 minutes

In this experiment, we will create a program that illustrates the service provided by the Academic Admin. Please refer to the following class diagram:

Student

nim:String name: String studyProgram: String className: String
Student (nim: String, name: String, studyProgram: String, className: String) void print()

2.2.1. Experiment Steps

Based on the class diagram, a **Student** class will be created in Java.

1. Create a new class named **Student**
2. Add the attributes as shown in the class diagram. Then, create the parametric constuctor as follows:

```
public Student(String nim, String name, String studyProgram, String
className){
        this.nim = nim;
        this.name = name;
        this.studyProgram = studyProgram;
        this.className = className;
    }
```

Provide **print()** method to display the whole data of student.

```
void print(){
        System.out.println(nim+" - "+name+" - "+studyProgram+" -
"+className);
    }
```

3. Copy the **Queue** class code from Experiment 1 to be reused in this experiment, and rename the class to **StudentQueue**. Since in Experiment 1, the data stored in the queue was only an **integer** array, while in this experiment the data used is Student object, you will need to modify the **StudentQueue** class accordingly.

```
Student[] data;
int front, rear, size, max;

public StudentQueue(int n){
        max = n;
        data = new Student[max];
        size = 0;
        front = 0;
        rear = -1;
}
```

4. Modify the **StudentQueue** class by changing the type of the data array from int[] to **Student**[], since in this case the data to be stored consists of **Student** objects. The modifications should be applied to the attributes, the **enqueue** method, and the **dequeue** method.

```
void enqueue(Student dt){
        if(isFull()){
            System.out.println("Queue is full!!!");
            return;
        }
        rear = (rear+1)%max;
        data[rear] = dt;
        size++;
        System.out.printf("%s is successfully added at index %d\n",
dt.name, rear);
}

Student dequeue(){
        if(isEmpty()){
            System.out.println("Queue is empty!!!");
            return null;
        }
        Student dt = data[front];
        front = (front+1)%max;
        size--;
        return dt;
}
```

5. Next, implement the **peek** and **print** methods to display the front-most data in the service queue and to display all the data in the service queue, respectively.

```
void peek(){
```

```
        if(!isEmpty()){
            System.out.println("Front data: ");
            data[front].print();
        }else{
            System.out.println("Queue is empty!!!");
        }
    }

    void print(){
        if(isEmpty()){
            System.out.println("Queue is empty!!!");
            return;
        }
        int i = front;
        while(i != rear){
            data[i].print();
            i = (i+1)%max;
        }
        data[i].print();
        System.out.println("Number of element: "+size);
    }
```

6. Next, create a new class named **StudentQueueMain** within the same package. In this class, define the **main** function and declare a **Scanner** object with the name **sc**.
7. Then, instantiate an **StudentQueue** object named **queue** with the parameter set to the maximum queue size (for example, 5).
8. Declare an integer variable named **choice** to store the user's menu selection.
9. Add the following code to loop through the menu options based on the input provided by the user:

```
public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        StudentQueue queue = new StudentQueue(5);
        int choice;
        do {
            System.out.println("\n=== Academic Service Menu ===");
            System.out.println("1. Enqueue Student");
            System.out.println("2. Dequeue Student");
            System.out.println("3. Display Front");
            System.out.println("4. Display All Data");
            System.out.println("5. Queue Size");
            System.out.println("0. Exit");
            System.out.print("Choose a menu: ");
            choice = sc.nextInt();
            sc.nextLine();
```

```
            switch (choice) {
                case 1:
                    System.out.print("NIM    : ");
                    String nim = sc.nextLine();
                    System.out.print("Name  : ");
                    String name = sc.nextLine();
                    System.out.print("Study Program : ");
                    String studyProgram = sc.nextLine();
                    System.out.print("Class : ");
                    String className = sc.nextLine();
                    Student std = new Student(nim, name, studyProgram,
className);
                    queue.enqueue(std);
                    break;
                case 2:
                    Student studentBeingServed = queue.dequeue();
                    if (studentBeingServed != null) {
                        System.out.println("Student being served:");
                        studentBeingServed.print();
                    }
                    break;
                case 3:
                    queue.peek();
                    break;
                case 4:
                    queue.print();
                    break;
                case 5:
                    System.out.println("Queue size: " + queue.size);
                    break;
                case 0:
                    System.out.println("Thanks!!");
                    break;
                default:
                    System.out.println("Invalid menu!!");
            }
        } while (choice != 0);
    }
```

10. Compile and run **StudentQueueMain** class and observe the result!

```java
package com.jobsheet11;

public class Student
{
    String  nim,
            name,
            studyProgram,
            className;

    public Student(String nim, String name, String studyProgram, String className)
    {
        this.nim = nim;
        this.name = name;
        this.studyProgram = studyProgram;
        this.className = className;
    }

    void print()
    {
        System.out.println(nim + " - " + name + " - " + studyProgram + " - " + className);
    }
}
```

```java
package com.jobsheet11;

public class StudentQueue
{
    Student[] data;
    int front,
        rear,
        size,
        max;

    public StudentQueue(int n)
    {
        max = n;
        data = new Student[max];
        size = 0;
        front = 0;
        rear = -1;
    }

    boolean isEmpty()
    {
        if (size == 0)
            return true;
        else
            return false;
    }

    boolean isFull()
    {
        if (size == max)
            return true;
        else
            return false;
    }

    void peek()
    {
        if (!isEmpty())
        {
            System.out.println(x: "Front data: ");
            data[front].print();
        }
        else
            System.out.println(x: "Queue is empty!!!");
    }

```

```java
    void print()
    {
        if (isEmpty())
        {
            System.out.println(x: "Queue is empty!!!");
            return;
        }
        else
        {
            int i = front;
            while (i != rear)
            {
                data[i].print();
                i = (i + 1) % max;
            }
            data[i].print();
            System.out.println("Number of element: " + size);
        }
    }

    void clear()
    {
        if (!isEmpty())
        {
            front = rear = -1;
            size = 0;
            System.out.println(x: "All data has been successfully removed!");
        }
        else
            System.out.println(x: "Queue is already empty!!!");
    }

    void enqueue(Student dt)
    {
        if (isFull())
        {
            System.out.println(x: "Queue is full!!!");
            return;
        }
        rear = (rear + 1) % max;
        data[rear] = dt;
        size++;
        System.out.printf(format: "%s is successfully added at index %d\n", dt.name, rear);
    }

    Student dequeue()
    {
        if (isEmpty())
        {
            System.out.println(x: "Queue is empty!!!");
            return null;
        }
        Student dt = data[front];
        front = (front + 1) % max;
        size--;
        return dt;
    }
}
```

```java
package com.jobsheet11;

import java.util.Scanner;

public class StudentQueueMain
{
    Run main | Debug main | Run | Debug
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        StudentQueue queue = new StudentQueue(n: 5);

        int choice;
        do
        {
            System.out.print
            (
                """
                \n=== Academic Service Menu ===
                1. Enqueue Student
                2. Dequeue Student
                3. Display Front
                4. Display All Data
                5. Queue Size
                0. Exit
                """
            );

```

```java
            System.out.print(s: "Choose a menu: ");
            choice = sc.nextInt();
            sc.nextLine();

            switch (choice)
            {
                case 1:
                    System.out.print(s: "NIM           : ");
                    String nim = sc.nextLine();
                    System.out.print(s: "Name          : ");
                    String name = sc.nextLine();
                    System.out.print(s: "Study Program : ");
                    String studyProgram = sc.nextLine();
                    System.out.print(s: "Class         : ");
                    String className = sc.nextLine();

                    Student std = new Student(nim, name, studyProgram, className);
                    queue.enqueue(std);
                    break;

                case 2:
                    Student studentBeingServed = queue.dequeue();
                    if (studentBeingServed != null)
                    {
                        System.out.println(x: "Student being served:");
                        studentBeingServed.print();
                    }
                    break;

                case 3:
                    queue.peek();
                    break;

                case 4:
                    queue.print();
                    break;

                case 5:
                    System.out.println("Queue size: " + queue.size);
                    break;

                case 0:
                    System.out.println(x: "Thanks!!");
                    sc.close();
                    break;

                default:
                    System.out.println(x: "Invalid menu!!");
            }
        } while (choice != 0);
    }
}
```

```
=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 1
NIM           : 111
Name          : Joseph
Study Program : TI
Class         : 1I
Joseph is successfully added at index 0

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 1
NIM           : 222
Name          : Brian
Study Program : TI
Class         : 1I
Brian is successfully added at index 1

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 1
NIM           : 333
Name          : Herco
Study Program : TI
Class         : 1I
Herco is successfully added at index 2
```

```
=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 1
NIM           : 444
Name          : Bima
Study Program : TI
Class         : 1I
Bima is successfully added at index 3

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 1
NIM           : 555
Name          : Kinan
Study Program : TI
Class         : 1I
Kinan is successfully added at index 4

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 1
NIM           : 6666
Name          : Nahda
Study Program : TI
Class         : 1I
Queue is full!!!
```

```
=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 5
Queue size: 5

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 4
111 - Joseph - TI - 1I
222 - Brian - TI - 1I
333 - Herco - TI - 1I
444 - Bima - TI - 1I
555 - Kinan - TI - 1I
Number of element: 5

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 3
Front data:
111 - Joseph - TI - 1I

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 2
Student being served:
111 - Joseph - TI - 1I
```

```
=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 5
Queue size: 4

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 4
222 - Brian - TI - 1I
333 - Herco - TI - 1I
444 - Bima - TI - 1I
555 - Kinan - TI - 1I
Number of element: 4

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 0
Thanks!!
```

2.2.2 Result Verification

Verify the output with the following:

```
=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
```

```
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 1
NIM    : 111
Name  : Joseph
Study Program : TI
Class : 1I
Joseph is successfully added at index 0

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 1
NIM    : 222
Name  : Brian
Study Program : TI
Class : 1I
Brian is successfully added at  index 1

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 1
NIM    : 333
Name  : Herco
Study Program : TI
Class : 1I
Herco is successfully added at  index 2

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 1
NIM    : 444
Name  : Bima
Study Program : TI
Class : 1I
Bima is successfully added at index 3

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 1
NIM    : 555
Name  : Kinan
Study Program : TI
Class : 1I
Kinan is successfully added at index 4

=== Academic Service Menu ===
```

```
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 1
NIM    : 6666
Name   : Nahda
Study Program : TI
Class : 1I
Queue is full!!!

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 5
Queue size: 5

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 4
111 - Joseph - TI - 1I
222 - Brian - TI - 1I
333 - Herco - TI - 1I
444 - Bima - TI - 1I
555 - Kinan - TI - 1I
Number of element: 5

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 3
Front data:
111 - Joseph - TI - 1I

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 2
Student being served:
111 - Joseph - TI - 1I

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
```

```
Choose a menu: 5
Queue size: 4

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 4
222 - Brian - TI - 1I
333 - Herco - TI - 1I
444 - Bima - TI - 1I
555 - Kinan - TI - 1I
Number of element: 4

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu:
```

2.2.3 Questions

1. What is the main difference between the **Queue** (experiment 1) and **StudentQueue** classes in terms of the **data type** they manage? Why is this change in data type important?

   a. *In the original Queue class, the queue stores primitive integers.*

   b. *In the StudentQueue class, the queue stores objects of type Student, which contain multiple attributes (nim, name, studyProgram, className).*

   This change is important because it allows the queue to handle complex data structures rather than simple numbers. It reflects a real-world scenario where queues often manage records or objects, not just primitive values.

2. How do the **enqueue** and **dequeue** methods in **StudentQueue** differ from those in **Queue**?
   a. *In the integer-based Queue, enqueue(int dt) and dequeue() work with primitive values, directly storing and returning integers.*
   b. *In StudentQueue, enqueue(Student dt) stores an entire Student object, and dequeue() returns a Student object.*

3. Examine the **constructor** of the **StudentQueue** class: why is the **front** attribute initialized to 0, whereas in the **Queue** class from Experiment 1, the **front** attribute is initialized to -1?

   a. *In the original Queue, front was initialised to -1 to mark the queue as empty, with no valid index.*

   b. *In StudentQueue, front is initialised to 0. This is because the implementation assumes that the logical start of the queue is at index 0 and uses size to determine whether the queue is empty.*

4. Modify the program by adding a new method called **viewRear()** in the **StudentQueue** class to

check the queue element at the **rear** position. Also, update the menu in the **StudentQueueMain** class by adding option 6: “Check rear of the queue,” so that the **viewRear()** method can be invoked.

```
    void viewRear()
    {
        if (!isEmpty())
        {
            System.out.println(x: "Rear data: ");
            data[rear].print();
        }
        else
            System.out.println(x: "Queue is empty!!!");
    }
```

```
System.out.print
(
    """
    \n=== Academic Service Menu ===
    1. Enqueue Student
    2. Dequeue Student
    3. Display Front
    4. Display All Data
    5. Queue Size
    6. Check rear of the queue
    0. Exit
    """
);
```

```
case 6:
    queue.viewRear();
    break;
```

2.3 Assignment

Time : 120 minutes

Create a queue-based program to simulate the approval queue for Student Course Registration Forms (KRS) by Academic Advisors (DPA). When a student joins the queue, they must first register their personal data (as in Experiment 2). Use queue classes similar to those used in Experiment1 and 2, and implement the following methods:

- Check if the queue is **empty**, check if the queue is **full**, and **clear** the queue.
- **Add** a student to the queue (enqueue), and process the KRS approval (dequeue) – each approval session processes **2 students at a time** (from the first two in the queue).

- Show **all students** in the queue, show the **first two** students in line, and show the **last student** in the queue.
- Print the **total number of students** in the queue, print the **number of students who have completed the KRS approval process**.
- The **maximum queue size is 10**, and each DPA can handle up to **30 students**. Display the number of students who have not yet completed the KRS approval process.

Draw the **class diagram** for the queue structure and implement all the required methods through a **menu-based interface** in the main function.

```java
package com.jobsheet11;

public class KRSStudent
{
    String  nim,
            name,
            studyProgram,
            className;

    public KRSStudent(String nim, String name, String studyProgram, String className)
    {
        this.nim = nim;
        this.name = name;
        this.studyProgram = studyProgram;
        this.className = className;

    }

    void print()
    {
        System.out.println(nim + "\t\t" + name + "\t\t\t" + studyProgram + "\t\t\t\t" + className);
    }
}
```

```java
package com.jobsheet11;

public class KRSQueue
{
    KRSStudent[] studentData;
    int front,
        rear,
        size,
        max,
        krsSize,
        krsRemain,
        krsMax;

    public KRSQueue(int n)
    {
        max = n;
        studentData = new KRSStudent[max];
        size = 0;
        front = -1;
        rear = -1;
        krsMax = 30;
        krsRemain = 30;
        krsSize = 0;
    }

    boolean isEmpty()
    {
        return size == 0;
    }

    boolean isFull()
    {
        return size == max;
    }

    boolean isFullKRS()
    {
        return krsSize == krsMax;
    }

```

```java
    void enqueue(KRSStudent dt)
    {
        if (isFull())
        {
            System.out.println(x: "Queue is full!!!");
            return;
        }

        rear = (rear + 1) % max;
        studentData[rear] = dt;
        size++;

        if (size == 1)
            front = rear;

        System.out.printf(format: "%s is successfully added at index %d\n", dt.name, rear);
    }

    KRSStudent dequeue()
    {
        if (isEmpty())
        {
            System.out.println(x: "Queue is empty!!!");
            return null;
        }
        else if (size == 1)
        {
            System.out.print
            (
                """
                The course registration process has failed!
                A minimum of 2 students is required in the queue.
                """
            );
            return null;
        }
        else if (isFullKRS())
        {
            System.out.print
            (
                """
                Course Registration System (KRS) error!
                The DPA has reached its maximum capacity.
                """
            );
            return null;
        }

```

```java
        KRSStudent data = studentData[front];

        size = size - 2;
        krsSize = krsSize + 2;
        krsRemain = krsRemain - 2;

        if (size == 0)
            front = rear = -1;
        else
            front = (front + 2) % max;

        return data;
    }

    void showAll()
    {
        if (isEmpty())
        {
            System.out.println(x: "Queue is Empty!!!");
            return;
        }

        int i = front;
            while (i != rear)
            {
                studentData[i].print();
                i = (i + 1) % max;
            }
        studentData[i].print();
    }

    void peek()
    {
        if (isEmpty())
        {
            System.out.println(x: "Queue is empty!!!");
            return;
        }
        else if (size == 1)
        {
            studentData[front].print();
            System.out.println(x: "There is only one student in the queue");
            return;
        }

        int secondIndex = (front + 1) % max;
        studentData[front].print();
        studentData[secondIndex].print();
    }

```

```java
    void viewRear()
    {
        if (!isEmpty())
            studentData[rear].print();
        else
            System.out.println(x: "Queue is empty!!!");
    }

    void check()
    {
        if (isEmpty())
            System.out.println(x: "Queue is empty!!!");
        else if (isFull())
            System.out.println(x: "Queue is full!!!");
        else
            System.out.printf(format: "There's %d students in Queue\n", size);
    }

    void clear()
    {
        if (!isEmpty())
        {
            front = rear = -1;
            size = 0;
            System.out.println(x: "All data has been successfully removed!");
        }
        else
            System.out.println(x: "Queue is already empty!!!");
    }
}
```

```java
package com.jobsheet11;

import java.util.Scanner;

public class KRSMain
{

    static void menu()
    {
        System.out.print
        (
            """

            ====================================
            COURSE REGISTRATION APPROVAL QUEUE SYSTEM
            ====================================
            1.  Add Student
            2.  Course Registration Approval Process (2 Students)
            3.  Show All Students in the Queue
            4.  Show 2 Top/Front Students
            5.  Show Students at the Back/Rear
            6.  Check Queue
            7.  Show the Number of Students in the Queue
            8.  Show the Number of Students who have Completed their Course Registration
            9.  Show the Number of Students who have not yet Registered
            10. Clear the Queue
            0.  Exit
            """
        );
    }

    static void StdChar()
    {
        System.out.println(x: "NIM     Name        Study Program   Class");
    }

```

```
Run | Debug | Run main | Debug main
public static void main(String[] args)
{
    Scanner sc = new Scanner(System.in);

    KRSQueue queue = new KRSQueue(n: 10);

    int choice;
    do
    {
        menu();
        System.out.print(s: "Choose a menu: ");
        choice = sc.nextInt();
        sc.nextLine();

        switch (choice)
        {
            case 1:
                System.out.print(s: "NIM           : ");
                String nim = sc.nextLine();
                System.out.print(s: "Name          : ");
                String name = sc.nextLine();
                System.out.print(s: "Study Program : ");
                String studyProgram = sc.nextLine();
                System.out.print(s: "Class         : ");
                String className = sc.nextLine();

                KRSStudent std = new KRSStudent(nim, name, studyProgram, className);
                queue.enqueue(std);
                break;

            case 2:
                int oldFront = queue.front;
                KRSStudent studentApprove = queue.dequeue();
                if (studentApprove != null)
                {
                    System.out.println(x: "2 Student has been aproved:");
                    StdChar();
                    int newFront = (oldFront + 1) % queue.max;
                    queue.studentData[oldFront].print();
                    queue.studentData[newFront].print();
                }
                break;
```

```java
                case 3:
                    StdChar();
                    queue.showAll();
                    break;

                case 4:
                    System.out.println(x: "2 Front Student is:");
                    StdChar();
                    queue.peek();
                    break;

                case 5:
                    System.out.println(x: "Rear data: ");
                    StdChar();
                    queue.viewRear();
                    break;

                case 6:
                    queue.check();
                    break;

                case 7:
                    System.out.println("Total Number of Student in Queue: " + queue.size);
                    break;

                case 8:
                    System.out.println("Total Number of Student has been approved: " + queue.krsSize);
                    break;

                case 9:
                    System.out.println("Total Number of Student has not been approved: " + queue.krsRemain);
                    break;

                case 10:
                    queue.clear();
                    break;

                case 0:
                    System.out.println(x: "Thank you!");
                    sc.close();
                    break;

                default:
                    System.out.println(x: "Input Invalid!!!");
            }
        }
        while (choice != 0);

    }
}
```

```
=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 1
NIM           : 111
Name          : A
Study Program : SIB
Class         : 1A
A is successfully added at index 0

=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 1
NIM           : 222
Name          : B
Study Program : SIB
Class         : 1A
B is successfully added at index 1
```

```
=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 1
NIM           : 333
Name          : C
Study Program : SIB
Class         : 1A
C is successfully added at index 2

=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 3
NIM     Name        Study Program   Class
111             A                       SIB                             1A
222             B                       SIB                             1A
333             C                       SIB                             1A
```

```
=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 4
2 Front Student is:
NIM     Name        Study Program   Class
111             A                       SIB                             1A
222             B                       SIB                             1A

=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 5
Rear data:
NIM     Name        Study Program   Class
333             C                       SIB                             1A

=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 2
2 Student has been aproved:
NIM     Name        Study Program   Class
111             A                       SIB                             1A
222             B                       SIB                             1A
```

```
=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 3
NIM     Name        Study Program   Class
333             C                       SIB                             1A

=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 1
NIM           : 444
Name          : D
Study Program : SIB
Class         : 1A
D is successfully added at index 3

=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 3
NIM     Name        Study Program   Class
333             C                       SIB                             1A
444             D                       SIB                             1A
```

```
=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 6
There's 2 students in Queue

=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 1
NIM           : 555
Name          : E
Study Program : SIB
Class         : 1A
E is successfully added at index 4

=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 3
NIM     Name        Study Program   Class
333             C                       SIB                             1A
444             D                       SIB                             1A
555             E                       SIB                             1A
```

```
=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 4
2 Front Student is:
NIM     Name        Study Program   Class
333             C                       SIB                             1A
444             D                       SIB                             1A

=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 5
Rear data:
NIM     Name        Study Program   Class
555             E                       SIB                             1A

=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 7
Total Number of Student in Queue: 3
```

```
=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 8
Total Number of Student has been approved: 2

=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 9
Total Number of Student has not been approved: 28

=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 10
All data has been successfully removed!
```

```
=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 3
NIM     Name        Study Program   Class
Queue is Empty!!!

=====================================
COURSE REGISTRATION APPROVAL QUEUE SYSTEM
=====================================
1.  Add Student
2.  Course Registration Approval Process (2 Students)
3.  Show All Students in the Queue
4.  Show 2 Top/Front Students
5.  Show Students at the Back/Rear
6.  Check Queue
7.  Show the Number of Students in the Queue
8.  Show the Number of Students who have Completed their Course Registration
9.  Show the Number of Students who have not yet Registered
10. Clear the Queue
0.  Exit
Choose a menu: 0
Thank you!
```

Github:

https://github.com/guramedadar/Praktikum-Algoritma-Struktur-Data/tree/main/src/com/jobsheet11